

SQL SELECT Combinations using AdventureWorks Database

Below is a structured list of commonly used and advanced combinations of `SELECT` with:

- `WHERE`
- `AND`
- `OR`
- `NOT`
- `IN`
- `BETWEEN`
- `LIKE`
- `GROUP BY`
- `HAVING`
- `ORDER BY`
- `TOP`
- `DISTINCT`
- `JOINS`
- `SUBQUERIES`
- `EXISTS`
- `ANY / ALL`
- `CASE`
- `CTE`
- `WINDOW FUNCTIONS`

using the AdventureWorks sample database.

1. Basic SELECT

```
SELECT *
```

```
FROM Person.Person;
```

2. SELECT Specific Columns

```
SELECT BusinessEntityID,
```

```
    FirstName,
```

```
LastName
```

```
FROM Person.Person;
```

3. SELECT with WHERE

```
SELECT *
```

```
FROM HumanResources.Employee
```

```
WHERE Gender = 'M';
```

4. WHERE with AND

```
SELECT *
```

```
FROM HumanResources.Employee
```

```
WHERE Gender = 'F'
```

```
AND MaritalStatus = 'M';
```

5. WHERE with OR

```
SELECT *
```

```
FROM HumanResources.Employee
```

```
WHERE JobTitle = 'Design Engineer'
```

```
OR JobTitle = 'Tool Designer';
```

6. WHERE with NOT

```
SELECT *
```

```
FROM HumanResources.Employee
```

```
WHERE NOT Gender = 'M';
```

7. WHERE with IN

```
SELECT *
```

```
FROM Production.Product
```

```
WHERE Color IN ('Red', 'Black', 'Silver');
```

8. WHERE with NOT IN

```
SELECT *
```

```
FROM Production.Product
```

```
WHERE Color NOT IN ('Red', 'Black');
```

9. WHERE with BETWEEN

```
SELECT *
```

```
FROM Sales.SalesOrderHeader
```

```
WHERE TotalDue BETWEEN 1000 AND 5000;
```

10. WHERE with NOT BETWEEN

```
SELECT *
```

```
FROM Sales.SalesOrderHeader
```

```
WHERE TotalDue NOT BETWEEN 1000 AND 5000;
```

11. WHERE with LIKE

```
SELECT *  
  
FROM Person.Person  
  
WHERE FirstName LIKE 'A%';
```

12. WHERE with NOT LIKE

```
SELECT *  
  
FROM Person.Person  
  
WHERE LastName NOT LIKE '%son';
```

13. DISTINCT

```
SELECT DISTINCT Color  
  
FROM Production.Product;
```

14. TOP

```
SELECT TOP 10 *  
  
FROM Sales.SalesOrderHeader  
  
ORDER BY TotalDue DESC;
```

15. ORDER BY ASC

```
SELECT *
```

```
FROM Production.Product
```

```
ORDER BY ListPrice ASC;
```

16. ORDER BY DESC

```
SELECT *
```

```
FROM Production.Product
```

```
ORDER BY ListPrice DESC;
```

17. GROUP BY

```
SELECT Color,
```

```
    COUNT(*) AS ProductCount
```

```
FROM Production.Product
```

```
GROUP BY Color;
```

18. GROUP BY with HAVING

```
SELECT Color,
```

```
    COUNT(*) AS ProductCount
```

```
FROM Production.Product
```

```
GROUP BY Color
```

```
HAVING COUNT(*) > 10;
```

19. GROUP BY with WHERE + HAVING

```
SELECT Color,  
  
        AVG(ListPrice) AS AvgPrice  
  
FROM Production.Product  
  
WHERE Color IS NOT NULL  
  
GROUP BY Color  
  
HAVING AVG(ListPrice) > 500;
```

20. Aggregate Functions

```
SELECT  
  
        MIN(ListPrice) AS MinPrice,  
  
        MAX(ListPrice) AS MaxPrice,  
  
        AVG(ListPrice) AS AvgPrice,  
  
        SUM(ListPrice) AS TotalPrice,  
  
        COUNT(*) AS TotalProducts  
  
FROM Production.Product;
```

21. INNER JOIN

```
SELECT p.FirstName,  
  
        p.LastName,  
  
        e.JobTitle
```

```
FROM Person.Person p

INNER JOIN HumanResources.Employee e

ON p.BusinessEntityID = e.BusinessEntityID;
```

22. LEFT JOIN

```
SELECT p.FirstName,

        soh.SalesOrderID

FROM Person.Person p

LEFT JOIN Sales.SalesOrderHeader soh

ON p.BusinessEntityID = soh.CustomerID;
```

23. RIGHT JOIN

```
SELECT p.Name,

        sod.OrderQty

FROM Production.Product p

RIGHT JOIN Sales.SalesOrderDetail sod

ON p.ProductID = sod.ProductID;
```

24. FULL OUTER JOIN

```
SELECT p.FirstName,

        c.CustomerID
```

```
FROM Person.Person p

FULL OUTER JOIN Sales.Customer c

ON p.BusinessEntityID = c.PersonID;
```

25. CROSS JOIN

```
SELECT p.Name,

        s.Name

FROM Production.Product p

CROSS JOIN Sales.SalesTerritory s;
```

26. SELF JOIN

```
SELECT e1.BusinessEntityID,

        e1.JobTitle,

        e2.BusinessEntityID AS ManagerID

FROM HumanResources.Employee e1

INNER JOIN HumanResources.Employee e2

ON e1.OrganizationLevel = e2.OrganizationLevel;
```

27. Simple Subquery

```
SELECT *

FROM Production.Product
```



```
WHERE ListPrice >

(

    SELECT AVG(ListPrice)

    FROM Production.Product

);
```

28. Subquery with IN

```
SELECT *

FROM Sales.Customer

WHERE CustomerID IN

(

    SELECT CustomerID

    FROM Sales.SalesOrderHeader

    WHERE TotalDue > 10000

);
```

29. Correlated Subquery

```
SELECT p1.ProductID,

    p1.Name,

    p1.ListPrice
```

```
FROM Production.Product p1

WHERE p1.ListPrice >

(

    SELECT AVG(p2.ListPrice)

    FROM Production.Product p2

    WHERE p1.Color = p2.Color

);
```

30. EXISTS

```
SELECT *

FROM Person.Person p

WHERE EXISTS

(

    SELECT 1

    FROM HumanResources.Employee e

    WHERE p.BusinessEntityID = e.BusinessEntityID

);
```

31. NOT EXISTS

```
SELECT *
```

```
FROM Person.Person p

WHERE NOT EXISTS

(

    SELECT 1

    FROM HumanResources.Employee e

    WHERE p.BusinessEntityID = e.BusinessEntityID

);
```

32. ANY

```
SELECT *

FROM Production.Product

WHERE ListPrice > ANY

(

    SELECT ListPrice

    FROM Production.Product

    WHERE Color = 'Red'

);
```

33. ALL

```
SELECT *
```

```
FROM Production.Product
```

```
WHERE ListPrice > ALL
```

```
(
```

```
    SELECT ListPrice
```

```
    FROM Production.Product
```

```
    WHERE Color = 'Red'
```

```
);
```

34. CASE Statement

```
SELECT Name,
```

```
    ListPrice,
```

```
    CASE
```

```
        WHEN ListPrice > 1000 THEN 'Expensive'
```

```
        WHEN ListPrice > 500 THEN 'Moderate'
```

```
        ELSE 'Cheap'
```

```
    END AS PriceCategory
```

```
FROM Production.Product;
```

35. Nested Subquery

```
SELECT *
```

```

FROM Sales.Customer

WHERE CustomerID IN

(

    SELECT CustomerID

    FROM Sales.SalesOrderHeader

    WHERE SalesPersonID IN

        (

            SELECT BusinessEntityID

            FROM Sales.SalesPerson

            WHERE SalesQuota > 300000

        )

);

```

36. Derived Table

```

SELECT *

FROM

(

    SELECT ProductID,

        AVG(OrderQty) AS AvgQty

```

```
FROM Sales.SalesOrderDetail

GROUP BY ProductID

) AS ProductSales

WHERE AvgQty > 5;
```

37. CTE (Common Table Expression)

```
WITH ProductCTE AS

(

    SELECT ProductID,

           Name,

           ListPrice

    FROM Production.Product

    WHERE ListPrice > 1000

)

SELECT *
```

```
FROM ProductCTE;
```

38. Recursive CTE

```
WITH Numbers AS
```

```
(
```

```
SELECT 1 AS Num

UNION ALL

SELECT Num + 1

FROM Numbers

WHERE Num < 10

)

SELECT *

FROM Numbers;
```

39. ROW_NUMBER()

```
SELECT ProductID,

        Name,

        ListPrice,

        ROW_NUMBER() OVER (ORDER BY ListPrice DESC) AS RowNum

FROM Production.Product;
```

40. RANK()

```
SELECT ProductID,

        Name,

        ListPrice,
```

```
RANK() OVER (ORDER BY ListPrice DESC) AS RankNum
```

```
FROM Production.Product;
```

41. DENSE_RANK()

```
SELECT ProductID,
```

```
    Name,
```

```
    ListPrice,
```

```
    DENSE_RANK() OVER (ORDER BY ListPrice DESC) AS DenseRankNum
```

```
FROM Production.Product;
```

42. PARTITION BY

```
SELECT ProductID,
```

```
    Color,
```

```
    ListPrice,
```

```
    AVG(ListPrice) OVER (PARTITION BY Color) AS AvgPriceByColor
```

```
FROM Production.Product;
```

43. SELECT with UNION

```
SELECT FirstName
```

```
FROM Person.Person
```

```
UNION
```



```
SELECT Name
```

```
FROM Production.Product;
```

44. UNION ALL

```
SELECT FirstName
```

```
FROM Person.Person
```

```
UNION ALL
```

```
SELECT Name
```

```
FROM Production.Product;
```

45. INTERSECT

```
SELECT BusinessEntityID
```

```
FROM HumanResources.Employee
```

```
INTERSECT
```

```
SELECT BusinessEntityID
```

```
FROM Person.Person;
```

46. EXCEPT

```
SELECT BusinessEntityID
```

```
FROM Person.Person
```

```
EXCEPT
```

```
SELECT BusinessEntityID
```

```
FROM HumanResources.Employee;
```

47. Complex Combination Query

```
SELECT
```

```
    p.Color,
```

```
    COUNT(*) AS ProductCount,
```

```
    AVG(p.ListPrice) AS AvgPrice
```

```
FROM Production.Product p
```

```
WHERE p.ListPrice > 100
```

```
AND p.Color IS NOT NULL
```

```
AND p.ProductID IN
```

```
(
```

```
    SELECT ProductID
```

```
    FROM Sales.SalesOrderDetail
```

```
    WHERE OrderQty > 2
```

```
)
```

```
GROUP BY p.Color
```

```
HAVING AVG(p.ListPrice) > 500
```

```
ORDER BY AvgPrice DESC;
```

48. Advanced Real-Time Reporting Query

```
SELECT
```

```
    YEAR(soh.OrderDate) AS OrderYear,
```

```
    MONTH(soh.OrderDate) AS OrderMonth,
```

```
    st.Name AS Territory,
```

```
    SUM(soh.TotalDue) AS Revenue
```

```
FROM Sales.SalesOrderHeader soh
```

```
INNER JOIN Sales.SalesTerritory st
```

```
ON soh.TerritoryID = st.TerritoryID
```

```
WHERE soh.Status = 5
```

```
GROUP BY
```

```
    YEAR(soh.OrderDate),
```

```
    MONTH(soh.OrderDate),
```

```
    st.Name
```

```
HAVING SUM(soh.TotalDue) > 100000
```

```
ORDER BY Revenue DESC;
```

49. Performance Optimized EXISTS Query

```
SELECT p.ProductID,  
  
       p.Name  
  
FROM Production.Product p  
  
WHERE EXISTS  
  
(  
  
  SELECT 1  
  
  FROM Sales.SalesOrderDetail sod  
  
  WHERE sod.ProductID = p.ProductID  
  
);
```

50. Complex Multi-Level Subquery

```
SELECT *  
  
FROM Sales.Customer  
  
WHERE CustomerID IN  
  
(  
  
  SELECT CustomerID  
  
  FROM Sales.SalesOrderHeader  
  
  WHERE TotalDue >  
  
  (  

```

```
SELECT AVG(TotalDue)

FROM Sales.SalesOrderHeader

)

);
```

Important SQL Execution Order

1. FROM
2. JOIN
3. WHERE
4. GROUP BY
5. HAVING
6. SELECT
7. DISTINCT
8. ORDER BY
9. TOP

Real-Time Interview Combination Example

```
SELECT

    d.Name AS DepartmentName,

    COUNT(e.BusinessEntityID) AS EmployeeCount,
```

```
AVG (pay.Rate) AS AvgSalary

FROM HumanResources.Employee e

INNER JOIN HumanResources.EmployeeDepartmentHistory edh

ON e.BusinessEntityID = edh.BusinessEntityID

INNER JOIN HumanResources.Department d

ON edh.DepartmentID = d.DepartmentID

INNER JOIN HumanResources.EmployeePayHistory pay

ON e.BusinessEntityID = pay.BusinessEntityID

WHERE e.Gender = 'M'

AND pay.Rate > 20

GROUP BY d.Name

HAVING COUNT (e.BusinessEntityID) > 5

ORDER BY AvgSalary DESC;
```

Common Interview Questions Based on SELECT

1. Difference between WHERE and HAVING
2. EXISTS vs IN
3. UNION vs UNION ALL
4. Correlated vs Non-Correlated Subquery
5. RANK vs DENSE_RANK vs ROW_NUMBER
6. GROUP BY execution flow
7. Order of SQL execution
8. Why aggregate functions cannot be used in WHERE
9. JOIN types with real-time examples
10. Subquery vs CTE vs Temp Table

